

Solutions of Question 1

Sequential Systems & Microprocessor

Note: The question asks “Answer any two”. Here all four sub-parts are solved for study purpose.

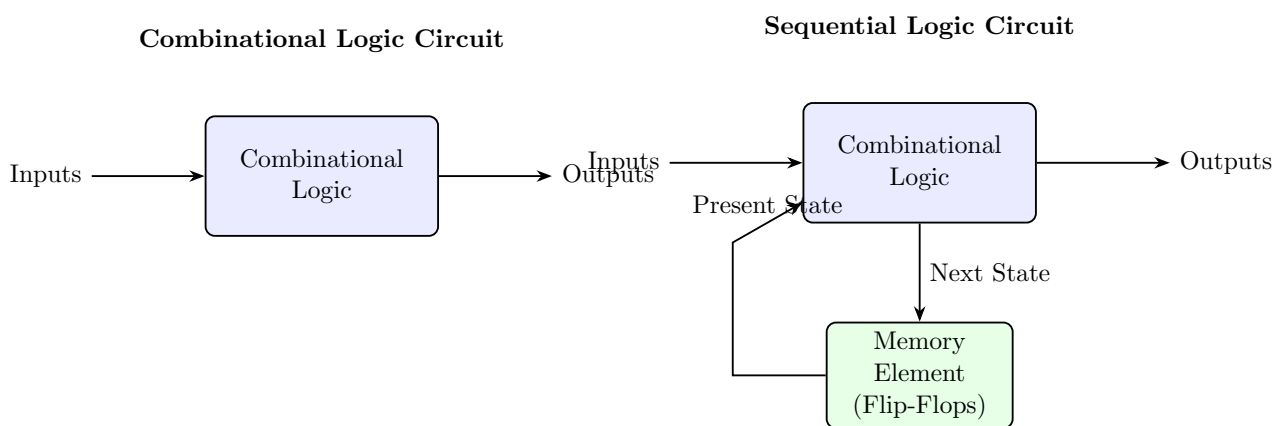
Q1(a) With the help of a schematic diagram discuss the basic differences between a Combinational logic circuit and a Sequential logic circuit.

Definition

A **combinational logic circuit** is a circuit whose output depends only on the **present input** values.

A **sequential logic circuit** is a circuit whose output depends on the **present input** values as well as the **previous state** of the circuit.

Schematic Diagrams



Basic Differences

Conclusion

Hence, the main difference is that a combinational circuit has **no memory**, whereas a sequential circuit has **memory** and its output depends on both **present input** and **past history/state**.

Sl. No.	Combinational Logic Circuit	Sequential Logic Circuit
1	Output depends only on present inputs.	Output depends on present inputs and previous state.
2	Does not require memory.	Requires memory elements such as flip-flops or latches.
3	No feedback path is present.	Feedback path is generally present.
4	No clock is required.	Usually operates with a clock signal.
5	Faster and simpler to design.	More complex because state transitions must be considered.
6	Examples: Adder, Subtractor, Encoder, Decoder, Multiplexer.	Examples: Counter, Register, Sequence Detector, FSM.

Q1(b) A fictitious flip-flop with two inputs A and B is designed such that for $AB = 00$ and 11 the output becomes 0 and 1, respectively. For $AB = 01$, flip-flop retains previous output while output complements for $AB = 10$. Determine the truth table and excitation table for this flip-flop.

Given Behaviour

The flip-flop behaves as follows:

$$AB = 00 \Rightarrow Q_{n+1} = 0$$

$$AB = 01 \Rightarrow Q_{n+1} = Q_n$$

$$AB = 10 \Rightarrow Q_{n+1} = \overline{Q_n}$$

$$AB = 11 \Rightarrow Q_{n+1} = 1$$

Truth Table (Characteristic Table)

Excitation Table

The excitation table shows what input combination AB is needed for a required transition from Q_n to Q_{n+1} .

Simplified Excitation Table

Sometimes the above table is also written using don't-care notation:

A	B	Q_n	Q_{n+1}
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Q_n	Q_{n+1}	Required Inputs AB
0	0	00 or 01
0	1	10 or 11
1	0	00 or 10
1	1	01 or 11

Final Answer

Thus, the required **truth table** and **excitation table** of the fictitious flip-flop are obtained as above.

Q_n	Q_{n+1}	A	B
0	0	0	X
0	1	1	X
1	0	X	0
1	1	X	1

Q1(c) Explain, with a schematic diagram, how does the State Transition Diagram for a Moore Machine differ from that for a Mealy Machine?

Basic Difference

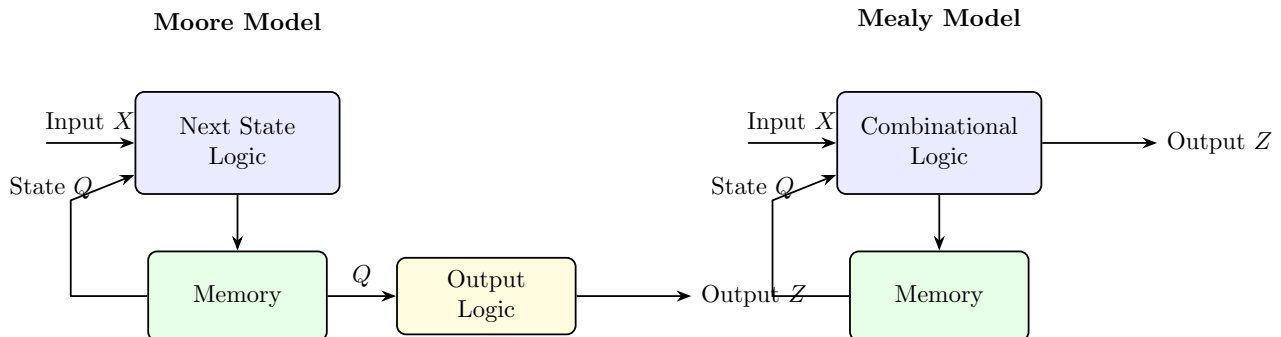
- In a **Moore machine**, the output depends only on the **present state**.

$$Z = f(Q)$$

- In a **Mealy machine**, the output depends on the **present state and present input**.

$$Z = f(Q, X)$$

Schematic Diagrams

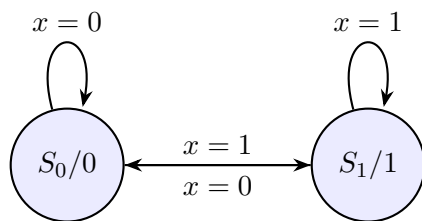


Difference in State Transition Diagram

- In a **Moore machine**, the **output is written inside the state**.
- In a **Mealy machine**, the **output is written on the transition arc** along with the input.

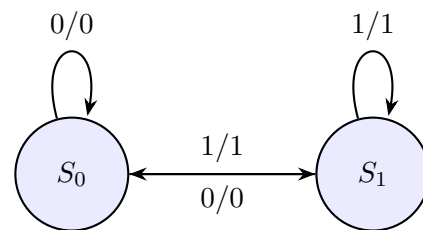
Illustrative State Diagrams

Moore State Diagram



Output is attached
to state

Mealy State Diagram



Output is attached
to transition

Conclusion

Therefore, the main difference is that in a **Moore machine**, output is associated with the **state**, whereas in a **Mealy machine**, output is associated with the **transition**.

Q1(d) What is a Finite State Machine? Obtain State Transition Diagram of SR-Flip-Flop.

Definition of Finite State Machine (FSM)

A **Finite State Machine (FSM)** is a mathematical model of a sequential circuit that has:

- a finite number of states,
- a set of inputs,
- a set of outputs,
- and rules for changing from one state to another.

In an FSM, the next state depends on the present state and input. The output depends either on:

- only the present state (Moore model), or
- both present state and input (Mealy model).

Truth Table of SR Flip-Flop

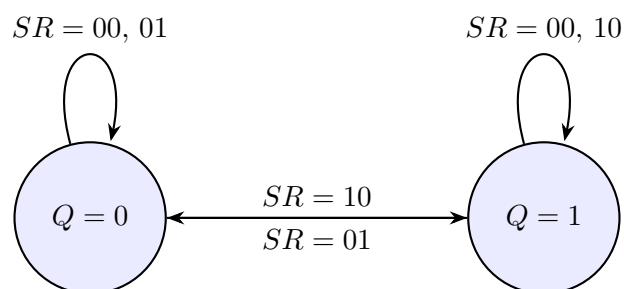
For an SR flip-flop:

S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	X	0
1	0	X	1
1	1	X	Invalid

State Transition Diagram of SR Flip-Flop

The SR flip-flop has two valid states:

$$Q = 0 \quad \text{and} \quad Q = 1$$



For $SR = 11$, the condition is invalid and therefore no valid transition is shown.

Explanation

- If $SR = 00$, the flip-flop **holds** its previous state.
- If $SR = 01$, the flip-flop is **reset**, so next state becomes $Q = 0$.
- If $SR = 10$, the flip-flop is **set**, so next state becomes $Q = 1$.
- If $SR = 11$, the state is **invalid** or forbidden.

Final Answer

A finite state machine is a sequential model with a finite number of states. The state transition diagram of an SR flip-flop consists of two states, $Q = 0$ and $Q = 1$, with transitions according to the valid input combinations $SR = 00, 01, 10$, while $SR = 11$ is invalid.

Q2(a) Operation of the Given Circuit

All flip-flops in the circuit are initially reset. Therefore,

$$A = 0, \quad Y = 0$$

Let,

$$A = \text{output of the first T flip-flop}$$

and

$$Y = \text{output of the second T flip-flop}$$

From the circuit diagram:

$$T_A = A + Y$$

and

$$T_Y = \bar{A}$$

For a T flip-flop, the next state equation is:

$$Q_{n+1} = T \oplus Q_n$$

Therefore,

$$A_{n+1} = T_A \oplus A$$

$$Y_{n+1} = T_Y \oplus Y$$

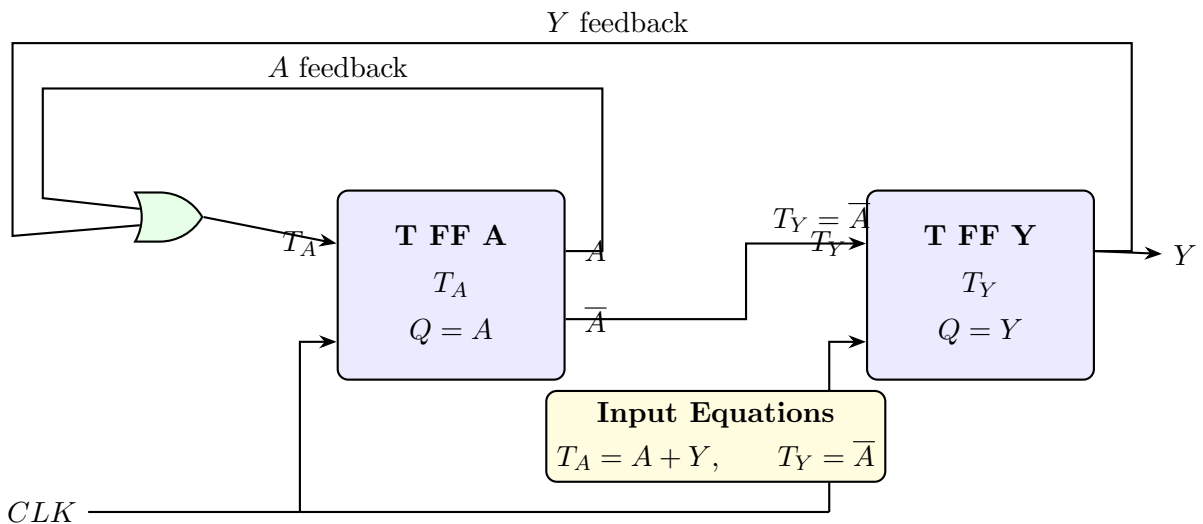
Hence,

$$A_{n+1} = (A + Y) \oplus A$$

and

$$Y_{n+1} = \bar{A} \oplus Y$$

Clean Schematic Diagram



State Table

For a T flip-flop:

$T = 0 \Rightarrow$ No change

$T = 1 \Rightarrow$ Toggle

The state table is obtained as follows:

Present State	$T_A = A + Y$	$T_Y = \overline{A}$	A_{n+1}	Y_{n+1}	Next State	
$AY = 00$	0	1	0	1	01	
$AY = 01$	1	1	1	0	10	
$AY = 10$	1	0	0	0	00	
$AY = 11$	1	0	0	1	01	

Operation of the Circuit

Since all flip-flops are initially reset, the initial state is:

$$AY = 00$$

Therefore, the circuit follows the sequence:

$$00 \rightarrow 01 \rightarrow 10 \rightarrow 00$$

Thus, the circuit behaves as a **mod-3 counter**. The unused state 11 goes to 01.

Q2(a)(ii) Timing Diagram

Initial condition:

$$A = 0, \quad Y = 0$$

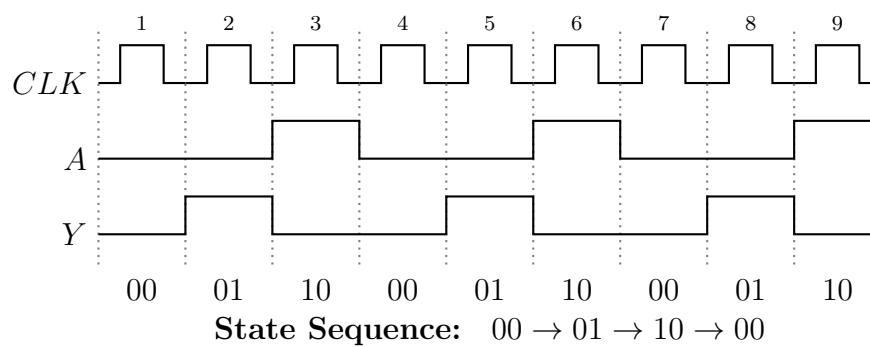
State sequence:

$$00, 01, 10, 00, 01, 10, 00, \dots$$

Therefore,

$$A : 0, 0, 1, 0, 0, 1, 0, \dots$$

$$Y : 0, 1, 0, 0, 1, 0, 0, \dots$$



Q2(b)(i) Excitation Table and Difference from Truth Table

Excitation Table

An **excitation table** of a flip-flop gives the required input values needed to produce a desired transition from present state Q_n to next state Q_{n+1} .

It answers the question:

What input should be applied to obtain a desired state change?

Truth Table

A **truth table** gives the next output for given input and present state.

It answers the question:

What will be the next state for a given input?

Difference Between Truth Table and Excitation Table

Point	Truth Table	Excitation Table
1	Gives next state for given inputs.	Gives required inputs for desired transition.
2	Input and present state are known.	Present state and next state are known.
3	Used for analysing a flip-flop.	Used for designing sequential circuits.
4	Form: Input, $Q_n \rightarrow Q_{n+1}$.	Form: $Q_n, Q_{n+1} \rightarrow$ Required input.

Conclusion

Truth table is mainly used for **analysis**, while excitation table is mainly used for **synthesis or design**.

Q2(b)(ii) Conversion of D Flip-Flop to SR Flip-Flop

To convert a D flip-flop into an SR flip-flop, the D input must be generated in such a way that the D flip-flop follows the behaviour of an SR flip-flop.

SR Flip-Flop Operation

The operation of an SR flip-flop is:

S	R	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	Invalid

For a D flip-flop,

$$Q_{n+1} = D$$

Hence, to obtain SR flip-flop behaviour from a D flip-flop, we must make:

$$D = Q_{n+1} \text{ of SR flip-flop}$$

Conversion Table

S	R	Q_n	Required $D = Q_{n+1}$	
0	0	0	0	
0	0	1	1	
0	1	0	0	
0	1	1	0	
1	0	0	1	
1	0	1	1	
1	1	0	X	
1	1	1	X	

Derivation of D Input

From the conversion table:

$$D = 1$$

for the cases:

$$S = 0, R = 0, Q_n = 1$$

and

$$S = 1, R = 0, Q_n = 0 \text{ or } 1$$

Therefore, the Boolean expression is:

$$D = S + \overline{R}Q_n$$

Hence,

$$\boxed{D = S + \overline{R}Q_n}$$

K-map for D Input

Variables are:

$$S, R, Q_n$$

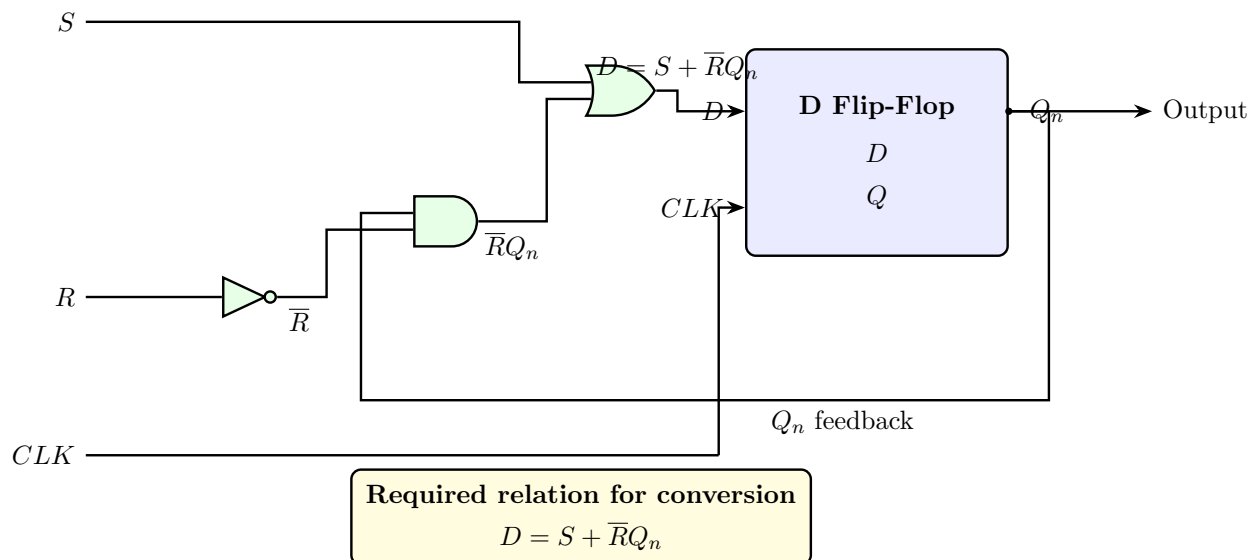
$S \backslash RQ_n$	00	01	11	10
$S = 0$	0	1	0	0
$S = 1$	1	1	X	X

From the K-map:

$$D = S + \overline{R}Q_n$$

$$D = S + \overline{R}Q_n$$

Circuit Diagram



Final Result

A D flip-flop can be converted to an SR flip-flop by applying the following signal to the D input:

$$D = S + \overline{R}Q_n$$

Thus, the D flip-flop behaves exactly like an SR flip-flop for all valid input combinations. The condition

$$S = R = 1$$

is invalid for an SR flip-flop, and therefore it is treated as a don't-care condition during simplification.

Analysis of Synchronous Counters

Solution: Q3(a): Counter Analysis and Cascading

Q3 (a) (i) Justify that the circuit is a Mod-3 counter

1. Identify the Flip-Flop Inputs From the circuit analysis, we have two JK Flip-Flops (A and B). The input logic is:

- **Flip-Flop A:** $J_A = \overline{B}$, $K_A = 1$
- **Flip-Flop B:** $J_B = A$, $K_B = 1$

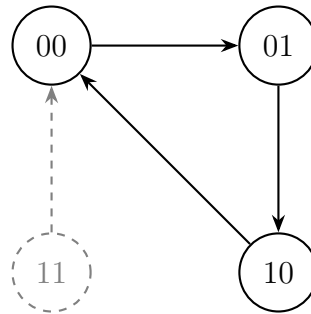
2. Next-State Equations Using $Q_{n+1} = J\overline{Q} + \overline{K}Q$:

- $A^+ = (\overline{B})\overline{A} + (0)A = \overline{AB}$
- $B^+ = (A)\overline{B} + (0)B = A\overline{B}$

3. State Transition Table

Present State		Inputs				Next State	
B	A	J_A	K_A	J_B	K_B	B^+	A^+
0	0	1	1	0	1	0	1
0	1	1	1	1	1	1	0
1	0	0	1	0	1	0	0
1	1	0	1	1	1	0	0

4. State Transition Diagram



The sequence is $00 \rightarrow 01 \rightarrow 10 \rightarrow 00$. Since there are **3 distinct states**, it is a **Mod-3 counter**.

Q3 (a) (ii) Proof of Mod-6 Conversion via Cascading

1. Concept of Cascading To convert a Mod-3 counter to Mod-6, we cascade a Mod-2 (Toggle) Flip-Flop (C).

- FF_C is triggered by the MSB (B) of the Mod-3 counter.
- $J_C = 1, K_C = 1$ (Toggle Mode).

2. Transition Analysis The Mod-2 stage toggles whenever the Mod-3 stage completes a full cycle (B goes $1 \rightarrow 0$).

Cycle	C (Mod-2)	B	A	Decimal
1	0	0	0	0
2	0	0	1	1
3	0	1	0	2
4	1	0	0	4
5	1	0	1	5
6	1	1	0	6

The total number of states is $3 \times 2 = 6$. Thus, it is a **Mod-6 counter**.

Q3 (a) (ii) Proof of Mod-6 Conversion via Cascading

1. Configuration of the 3rd Flip-Flop (Let's call it FF C)

- The clock input of FF C is driven by the B output of the Mod-3 counter.
- The inputs to FF C are $J_C = 1$ and $K_C = 1$.
- Because $J = K = 1$, FF C operates in **toggle mode**. It will flip its state ($0 \rightarrow 1$ or $1 \rightarrow 0$) every time it receives an active clock edge from output B .

2. Timing Analysis of Output B Let's look at the sequence of the B bit in our Mod-3 counter (BA):

Sequence: $00 \rightarrow 01 \rightarrow 10 \rightarrow 00$

The bit B transitions as follows:

1. $0 \rightarrow 0$ (No edge)
2. $0 \rightarrow 1$ (Positive Edge)
3. $1 \rightarrow 0$ (**Negative Edge**)

Assuming the flip-flops are negative edge-triggered (as indicated by the bubbles on the clock inputs in the original schematic), Flip-Flop C will toggle exactly **once per complete cycle** of the Mod-3 counter (specifically, when the Mod-3 counter transitions from 10 back to 00).

3. Combined State Sequence (CBA) Let's trace the combined states of all three Flip-Flops. FF C acts as the MSB.

Clock Pulse	Mod-3 State (BA)		FF C State	Combined State (CBA)
Initial	0	0	0	000 (0 in decimal)
1	0	1	0	001 (1 in decimal)
2	1	0	0	010 (2 in decimal)
3	0	0	1	100 (4 in decimal) – C toggles because B went $1 \rightarrow 0$
4	0	1	1	101 (5 in decimal)
5	1	0	1	110 (6 in decimal)
6	0	0	0	000 (0 in decimal) – C toggles because B went $1 \rightarrow 0$

4. Conclusion The combined system cycles through the sequence:

$$000 \rightarrow 001 \rightarrow 010 \rightarrow 100 \rightarrow 101 \rightarrow 110 \rightarrow 000$$

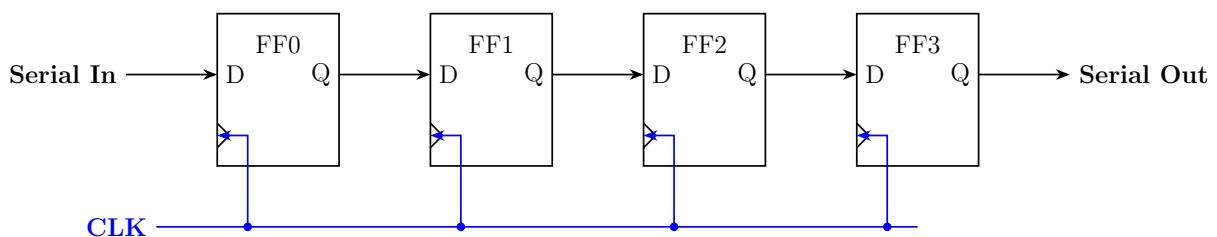
By cascading a Mod-2 counter (the toggle Flip-Flop C) to the output of a Mod-3 counter, the total number of states multiplies ($3 \times 2 = 6$). Because the system has exactly **6 distinct states**, it has successfully become a **Mod-6 counter**.

Part (b): Shift Register Configurations

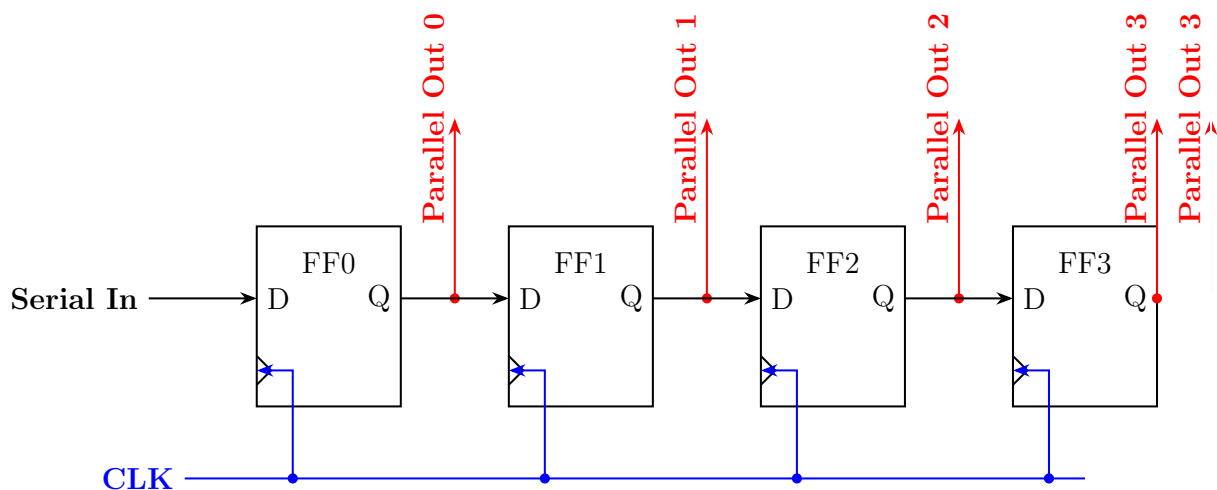
Shift registers are categorized by their data I/O methods. The four commercial configurations are:

- **SISO (Serial-In, Serial-Out):** Data enters and leaves sequentially.
- **SIPO (Serial-In, Parallel-Out):** Serial data is converted to parallel format.
- **PISO (Parallel-In, Serial-Out):** Parallel data is converted to serial format.
- **PIPO (Parallel-In, Parallel-Out):** Used for high-speed data buffering.

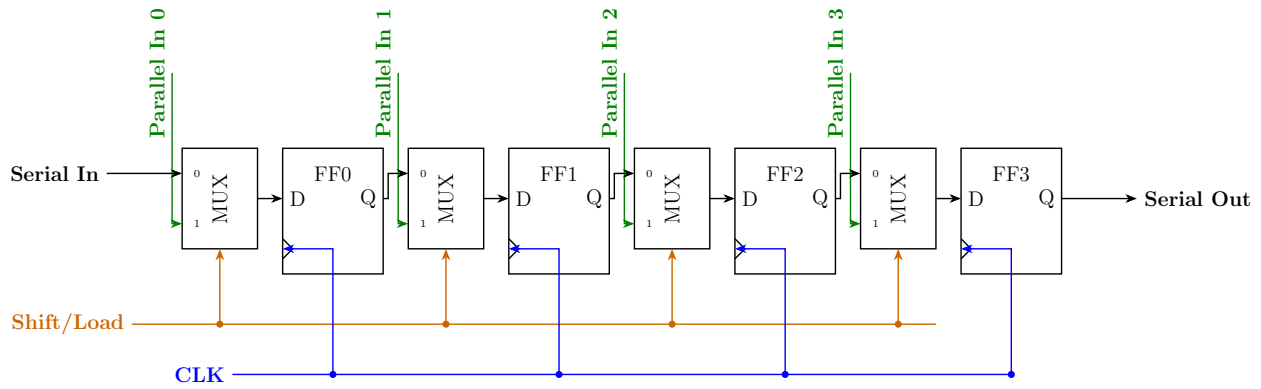
1. SISO (Serial-In, Serial-Out) Configuration



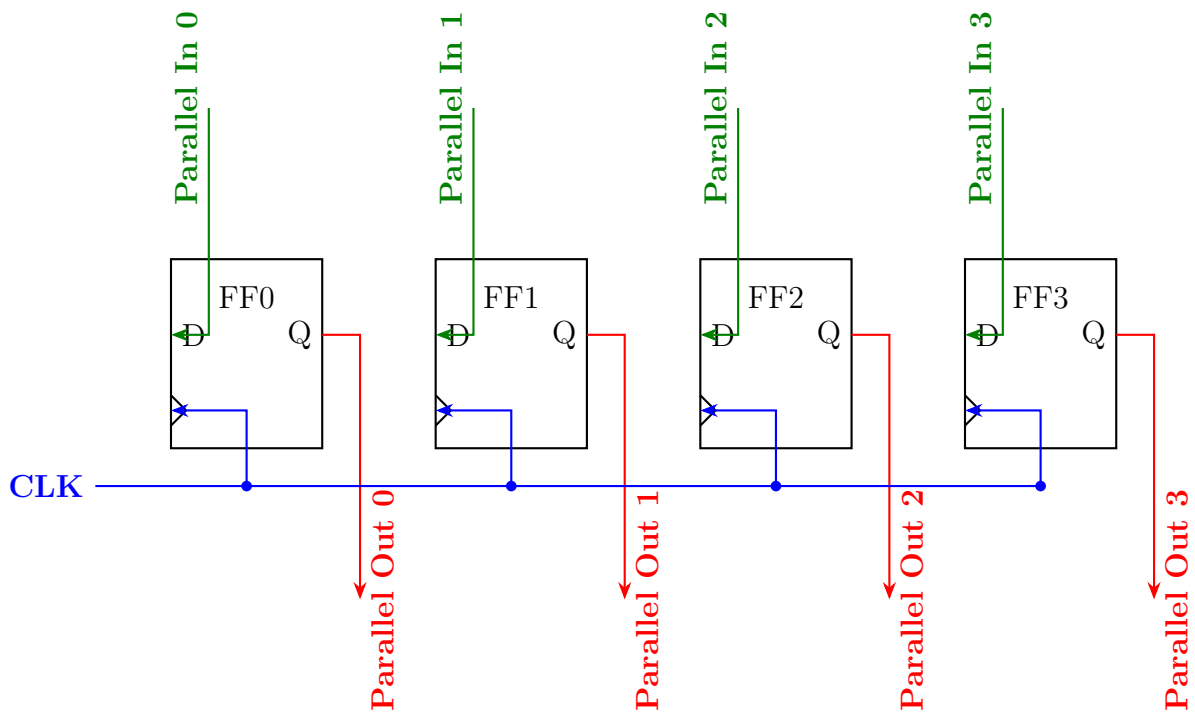
2. SIPO (Serial-In, Parallel-Out) Configuration



3. PISO (Parallel-In, Serial-Out) Configuration



4. PIPO (Parallel-In, Parallel-Out) Configuration

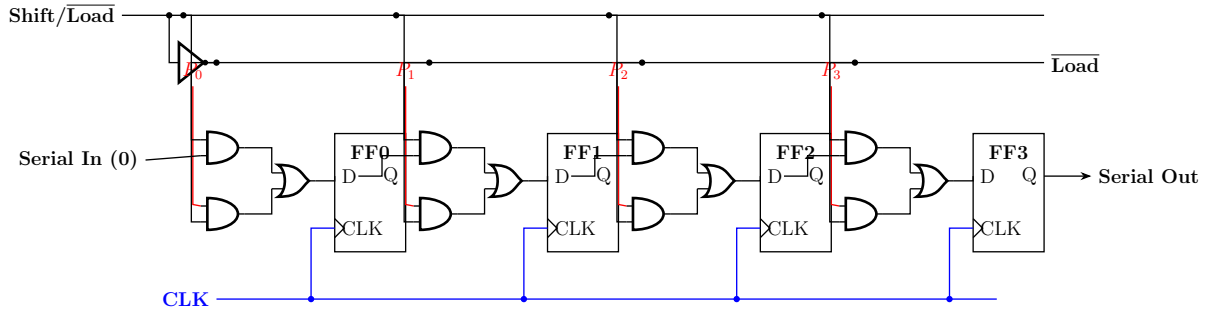


Shift Register Implementations

Q(c) 4-Bit Parallel-In Serial-Out (PISO) Shift Right Register

1. Schematic Diagram

The circuit is realized using four D flip-flops. To select between loading parallel data and shifting data to the right, a combinational logic circuit (acting as a 2-to-1 Multiplexer) is placed before the D input of each flip-flop. This is controlled by a master **Shift/Load** line.



2. Functional Description

A Parallel-In Serial-Out (PISO) register accepts a multi-bit word simultaneously on its parallel input lines and then shifts that data out one bit at a time on a single serial output line. The mode of operation is dictated by the **Shift/Load** control line. All flip-flops are triggered synchronously by the same clock edge.

Mode 1: Parallel Loading ($\text{Shift}/\overline{\text{Load}} = 0$)

- When the control line is set to 0, the inverted line ($\overline{\text{Load}}$) becomes 1.
- This logic HIGH enables the lower AND gates connected to the parallel inputs (P_0, P_1, P_2, P_3).
- Simultaneously, the logic 0 on the main control line disables the upper AND gates, blocking any data from the preceding flip-flops.
- The parallel data passes through the active AND gates, through the OR gates, and waits at the D inputs.
- **Action:** On the very first active clock edge, all 4 bits of parallel data are loaded simultaneously into the flip-flops ($Q_0 = P_0, Q_1 = P_1$, etc.).

Mode 2: Serial Shifting ($\text{Shift}/\overline{\text{Load}} = 1$)

- To extract the data serially, the control line is switched to 1. The inverted line becomes 0.
- The logic 0 disables the lower AND gates, isolating the flip-flops from the external parallel inputs.
- The logic HIGH enables the upper AND gates. This establishes a connection between the Q output of one flip-flop and the D input of the adjacent right-hand flip-flop.
- **Action:** On each subsequent clock edge, the stored bits shift one position to the right ($D_1 \leftarrow Q_0, D_2 \leftarrow Q_1, D_3 \leftarrow Q_2$). The LSB (Q_3) is outputted to the Serial Out pin. It requires 3 clock pulses to completely shift a 4-bit word out after it has been loaded.

Counter Classifications

Q4 (a) (i) Synchronous vs. Asynchronous Counters

The major differences between Synchronous and Asynchronous (Ripple) counters are summarized below:

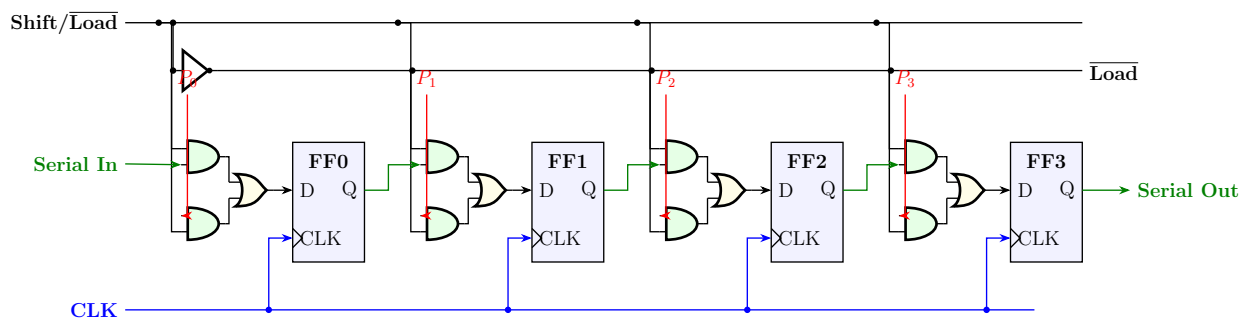
Synchronous Counters	Asynchronous (Ripple) Counters
All flip-flops are triggered simultaneously by the same global clock signal.	Only the first flip-flop is triggered by the external clock. Subsequent flip-flops are triggered by the output of the preceding flip-flop.
Faster operation. There is no cumulative propagation delay since all states change at once.	Slower operation. The propagation delays of each flip-flop add up (ripple effect), limiting the maximum clock frequency.
Circuit design becomes highly complex for higher states, requiring significant combinational logic gates to determine next-state inputs.	Circuit design is very simple and straightforward. It requires minimal or no extra logic gates for basic up/down counting.
Does not suffer from decoding errors or transient states (glitches).	Prone to decoding errors (glitches) due to the slight delay between state transitions.

Shift Register Implementations

Q4(a)(ii) 4-Bit Parallel-In Serial-Out (PISO) Shift Right Register

1. Schematic Diagram

The circuit is realized using four D flip-flops. To select between loading parallel data and shifting data to the right, a combinational logic circuit (acting as a 2-to-1 Multiplexer) is placed before the D input of each flip-flop. This is controlled by a master **Shift/Load** line.



2. Functional Description

A Parallel-In Serial-Out (PISO) register accepts a multi-bit word simultaneously on its parallel input lines and then shifts that data out one bit at a time on a single serial output line.

The mode of operation is dictated by the **Shift/Load** control line. All flip-flops are triggered synchronously by the same clock edge.

2. Functional Description

A Parallel-In Serial-Out (PISO) register accepts a multi-bit word simultaneously on its parallel input lines and then shifts that data out one bit at a time on a single serial output line. The mode of operation is dictated by the **Shift/Load** control line. All flip-flops are triggered synchronously by the same clock edge.

Mode 1: Parallel Loading ($\text{Shift}/\overline{\text{Load}} = 0$)

- When the control line is set to 0, the inverted line ($\overline{\text{Load}}$) becomes 1.
- This logic HIGH enables the lower AND gates connected to the parallel inputs (P_0, P_1, P_2, P_3).
- Simultaneously, the logic 0 on the main control line disables the upper AND gates, blocking any data from the preceding flip-flops.
- The parallel data passes through the active AND gates, through the OR gates, and waits at the D inputs.
- **Action:** On the very first active clock edge, all 4 bits of parallel data are loaded simultaneously into the flip-flops ($Q_0 = P_0, Q_1 = P_1$, etc.).

Mode 2: Serial Shifting ($\text{Shift}/\overline{\text{Load}} = 1$)

- To extract the data serially, the control line is switched to 1. The inverted line becomes 0.
- The logic 0 disables the lower AND gates, isolating the flip-flops from the external parallel inputs.
- The logic HIGH enables the upper AND gates. This establishes a connection between the Q output of one flip-flop and the D input of the adjacent right-hand flip-flop.
- **Action:** On each subsequent clock edge, the stored bits shift one position to the right ($D_1 \leftarrow Q_0, D_2 \leftarrow Q_1, D_3 \leftarrow Q_2$). The LSB (Q_3) is outputted to the Serial Out pin. It requires 3 clock pulses to completely shift a 4-bit word out after it has been loaded.

3. Serial Shifting Trace Example

To clearly illustrate the serial shifting process, let's trace the loading and shifting of a 4-bit data word, for example, **1011** ($P_0 = 1, P_1 = 0, P_2 = 1, P_3 = 1$). Assume the Serial Input line is tied to Logic 0.

Clock Pulse	Mode	Shift/ $\overline{\text{Load}}$	Q_0	Q_1	Q_2	Q_3	Serial Output (Q_3)
Initial	-	-	0	0	0	0	0
1	Parallel Load	0	1	0	1	1	1 (Bit 0)
2	Shift Right 1	1	0	1	0	1	1 (Bit 1)
3	Shift Right 2	1	0	0	1	0	0 (Bit 2)
4	Shift Right 3	1	0	0	0	1	1 (Bit 3)

As shown in the table, it takes 1 clock pulse to load the data, and exactly 3 additional clock pulses to completely shift out the remaining bits of the 4-bit word serially.

Design a synchronous sequential circuit having the state transition diagram, as shown in Figure Q4(b), with the help of JK flip-flops.

1. State Transition & Excitation Table

Let the two states be A (MSB) and B (LSB), and let the external input be X . By tracing the provided state transition diagram, we can map the present state and input to the next state, and derive the required JK flip-flop inputs using the standard JK excitation table.

Present State		Input	Next State		Required FF Inputs			
A	B	X	A^+	B^+	J_A	K_A	J_B	K_B
0	0	0	0	0	0	X	0	X
0	0	1	0	1	0	X	1	X
0	1	0	1	0	1	X	X	1
0	1	1	0	1	0	X	X	0
1	0	0	1	0	X	0	0	X
1	0	1	1	1	X	0	1	X
1	1	0	1	1	X	0	X	0
1	1	1	0	0	X	1	X	1

2. K-Map Simplifications

We plot the required inputs on 3-variable Karnaugh maps (where columns are BX) to derive the minimal Boolean expressions for the logic gates.

K-map for J_A

$A \setminus BX$	00	01	11	10
0	0	0	0	1
1	X	X	X	X

$$J_A = B\bar{X}$$

K-map for K_A

$A \setminus BX$	00	01	11	10
0	X	X	X	X
1	0	0	1	0

$$K_A = BX$$

K-map for J_B

$A \setminus BX$	00	01	11	10
0	0	1	X	X
1	0	1	X	X

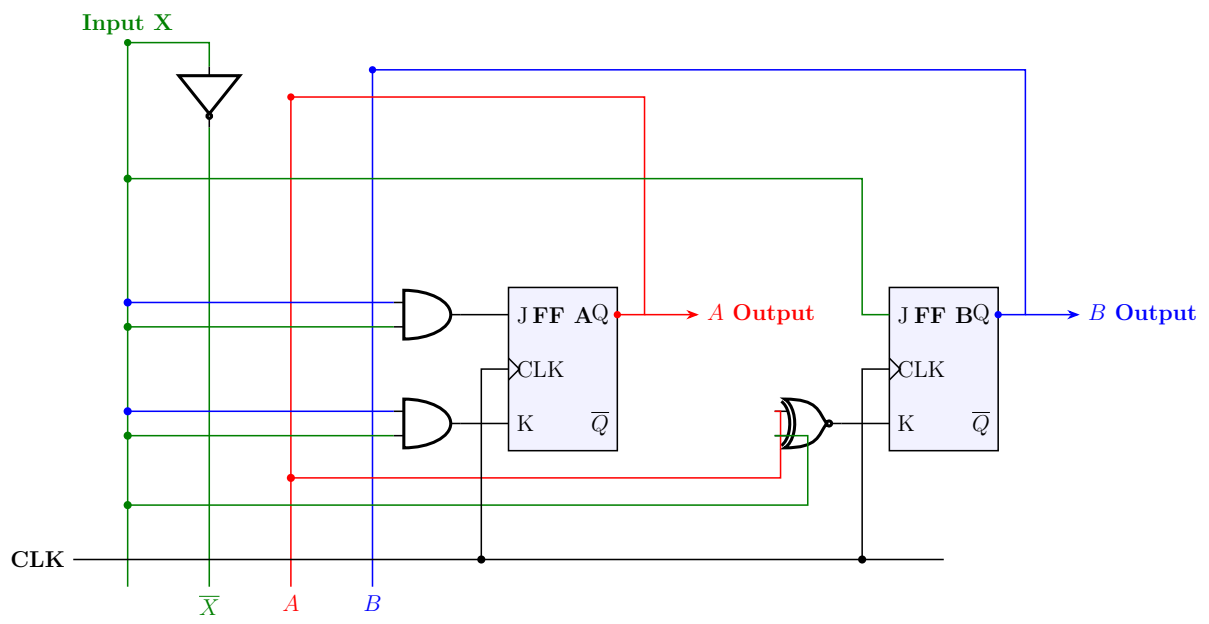
$$J_B = X$$

K-map for K_B

$A \setminus BX$	00	01	11	10
0	X	X	0	1
1	X	X	1	0

$$K_B = \bar{A}\bar{X} + AX = A \odot X$$

3. Logic Circuit Diagram



Q5(a) What is a State Transition Diagram? Show how the State Transition Diagram for a Mealy Model as shown in Figure Q5(a) can be converted to the equivalent Moore Model diagrams.

Definition of State Transition Diagram

A **State Transition Diagram** is a graphical representation of the behaviour of a sequential circuit or finite state machine (FSM). In this diagram:

- each **state** is represented by a circle,
- each **transition** is represented by a directed arrow,
- the condition for transition is written on the arrow.

For a:

- **Mealy machine:** transition label is of the form

input/output

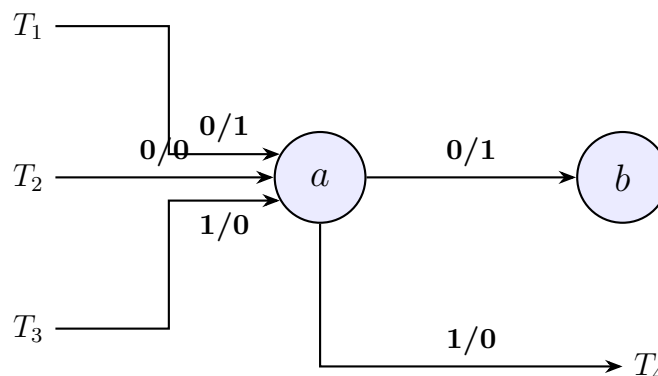
- **Moore machine:** output is written inside the state, and the transition label contains only the input.

Important Difference

In a **Mealy model**, output is associated with the **transition**.

In a **Moore model**, output is associated with the **state**.

Given Mealy State Diagram



Rule for Converting Mealy Model to Moore Model

To convert a Mealy model into a Moore model, we use the following rule:

- If a Mealy state receives transitions carrying **different output values**, then that state must be **split into multiple Moore states**.

- Each split state will have one fixed output.

From the given Mealy diagram:

- State a receives transitions with outputs 1 and 0. Therefore, a must be split into two Moore states:

$$a_1 \text{ (output 1),} \quad a_0 \text{ (output 0)}$$

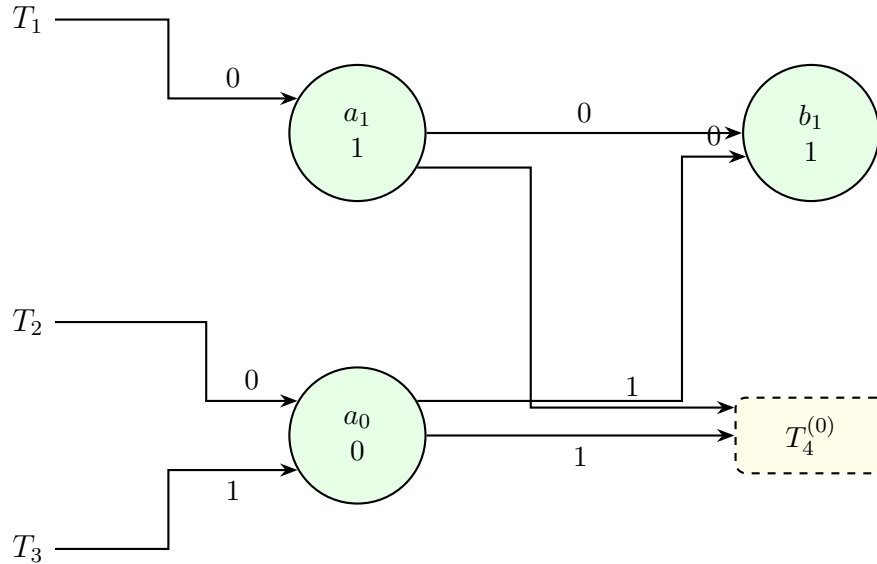
- State b receives only one transition with output 1. Therefore, only one Moore state is needed:

$$b_1 \text{ (output 1)}$$

Also, the transition from a to T_4 has output 0, so the destination reached through T_4 must correspond to a Moore state having output 0. We denote this by:

$$T_4^{(0)}$$

Equivalent Moore Model Diagram



Explanation of the Converted Moore Diagram

In the Moore model:

- a_1 is the version of state a with output 1,
- a_0 is the version of state a with output 0,
- b_1 is the version of state b with output 1.

Thus, the Mealy transitions are converted as follows:

$$T_1 \xrightarrow{0/1} a \implies T_1 \xrightarrow{0} a_1$$

$$T_2 \xrightarrow{0/0} a \implies T_2 \xrightarrow{0} a_0$$

$$T_3 \xrightarrow{1/0} a \implies T_3 \xrightarrow{1} a_0$$

$$a \xrightarrow{0/1} b \implies a_1 \xrightarrow{0} b_1, \quad a_0 \xrightarrow{0} b_1$$

$$a \xrightarrow{1/0} T_4 \implies a_1 \xrightarrow{1} T_4^{(0)}, \quad a_0 \xrightarrow{1} T_4^{(0)}$$

Final Answer

A state transition diagram is the graphical representation of the states and transitions of a sequential machine.

For conversion from Mealy to Moore model:

- state a is split into a_1 and a_0 ,
- state b becomes b_1 ,
- outputs are shifted from transitions to states.

Hence, the equivalent Moore model is obtained by creating separate states corresponding to different output values associated with the destination states.

Sequence Generator Design

Solution: (b) Generating "1101" using a Modulo-4 Counter

1. Memory Element Requirement

The target sequence "1101" has a length of 4. To generate a 4-bit repeating sequence, the system must cycle through 4 distinct states. A synchronous **Modulo-4 counter** naturally provides exactly 4 states (00, 01, 10, 11) using 2 JK flip-flops ($2^2 = 4$). Therefore, 2 memory elements are the minimum required to fulfill this condition.

2. Modulo-4 Counter Setup

We will use a standard 2-bit synchronous up-counter consisting of FF_1 (MSB, Q_1) and FF_0 (LSB, Q_0). The standard excitation logic for a synchronous Mod-4 counter is:

- $J_0 = 1, \quad K_0 = 1$ (Toggles every clock cycle)
- $J_1 = Q_0, \quad K_1 = Q_0$ (Toggles only when Q_0 is 1)

3. Output Logic Mapping

We must map the internal states of the Mod-4 counter to the desired output sequence $Y = 1, 1, 0, 1$.

Clock Pulse	State Q_1	State Q_0	Desired Output (Y)
0	0	0	1
1	0	1	1
2	1	0	0
3	1	1	1

4. Boolean Simplification for Output Y

From the truth table, the sum-of-minterms for Y is $\Sigma m(0, 1, 3)$. We can solve this using boolean algebra or a simple 2-variable K-map:

$$Y = \overline{Q_1}\overline{Q_0} + \overline{Q_1}Q_0 + Q_1Q_0$$

$$Y = \overline{Q_1}(\overline{Q_0} + Q_0) + Q_1Q_0$$

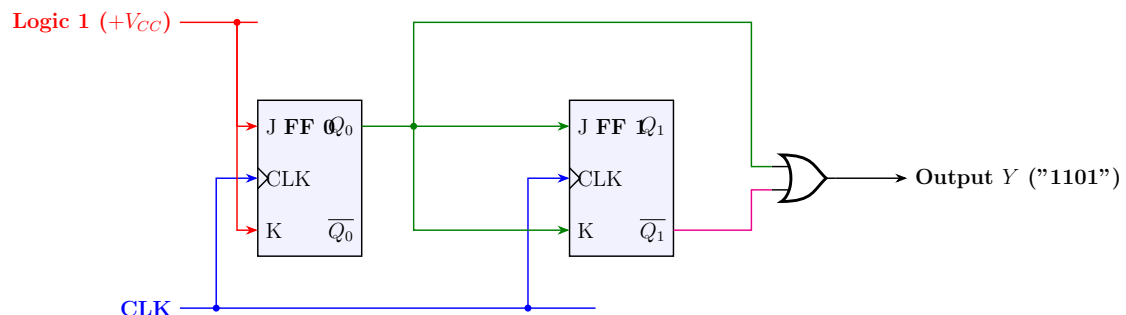
$$Y = \overline{Q_1} + Q_1Q_0$$

By applying the distributive law ($A + \overline{A}B = A + B$):

$$Y = \overline{Q_1} + Q_0$$

This is a highly optimized result! We only need a single OR gate taking inputs from the $\overline{Q}$ output of FF_1 and the Q output of FF_0 .

5. Logic Circuit Diagram



Q(5)(c) System Definitions & State Modeling

Before drawing the diagrams, let us define the variables to ensure clarity:

- **States (Money Accumulated):**

- S_0 : Rs. 0 accumulated
- S_1 : Re. 1 accumulated
- S_2 : Rs. 2 accumulated

- **Inputs (I, J):**

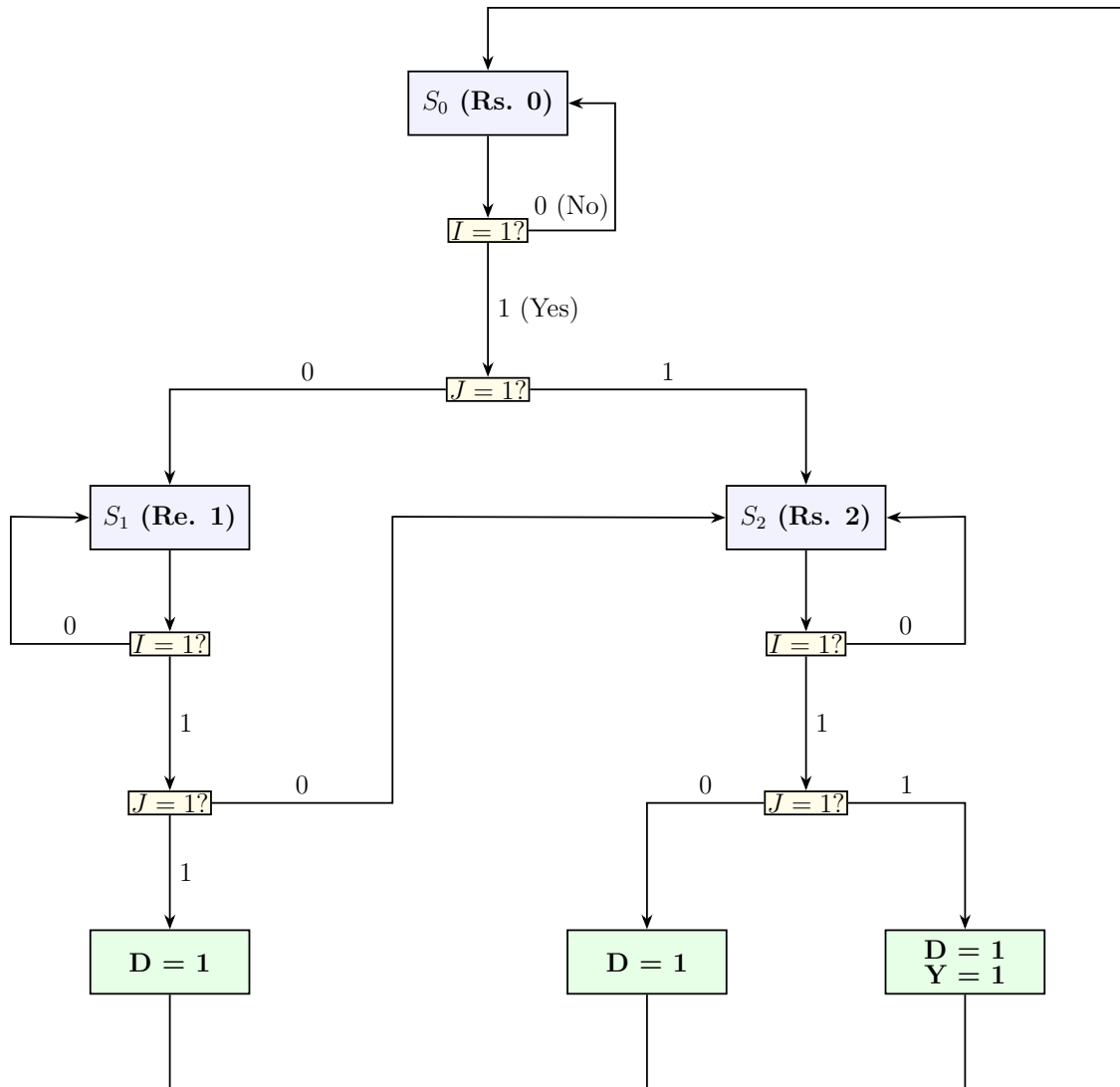
- $0X$: No coin inserted
- 10 : Re. 1 coin inserted
- 11 : Rs. 2 coin inserted

- **Outputs (D, Y):**

- D : Product Delivery (1 = Deliver, 0 = Wait)
- Y : Coin Return Mechanism (1 = Return Re. 1, 0 = Wait)

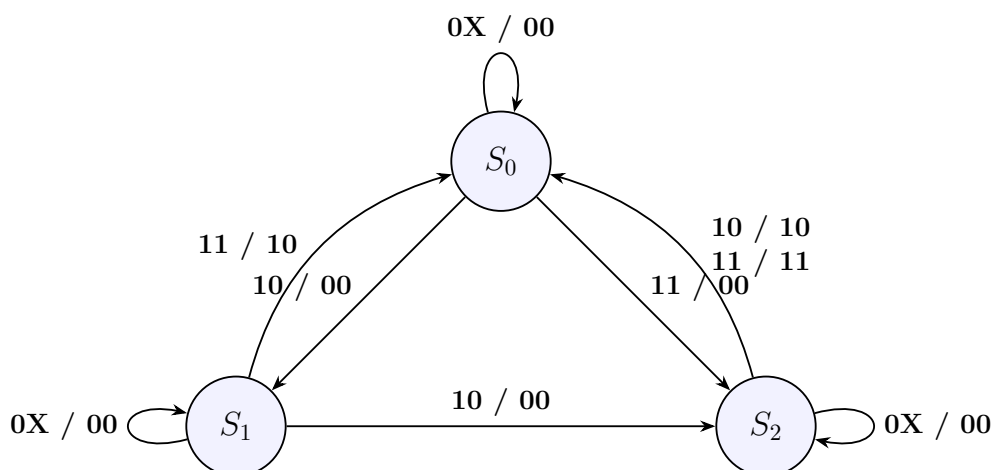
Note: Because this is a Mealy model, outputs are determined during the transition between states rather than being tied to the state itself.

Solution: (i) ASM Chart for the Mealy Model



Solution: (ii) Corresponding State Transition Diagram

The state transition diagram uses the format **Input / Output** on the edges, specifically mapping IJ/DY .



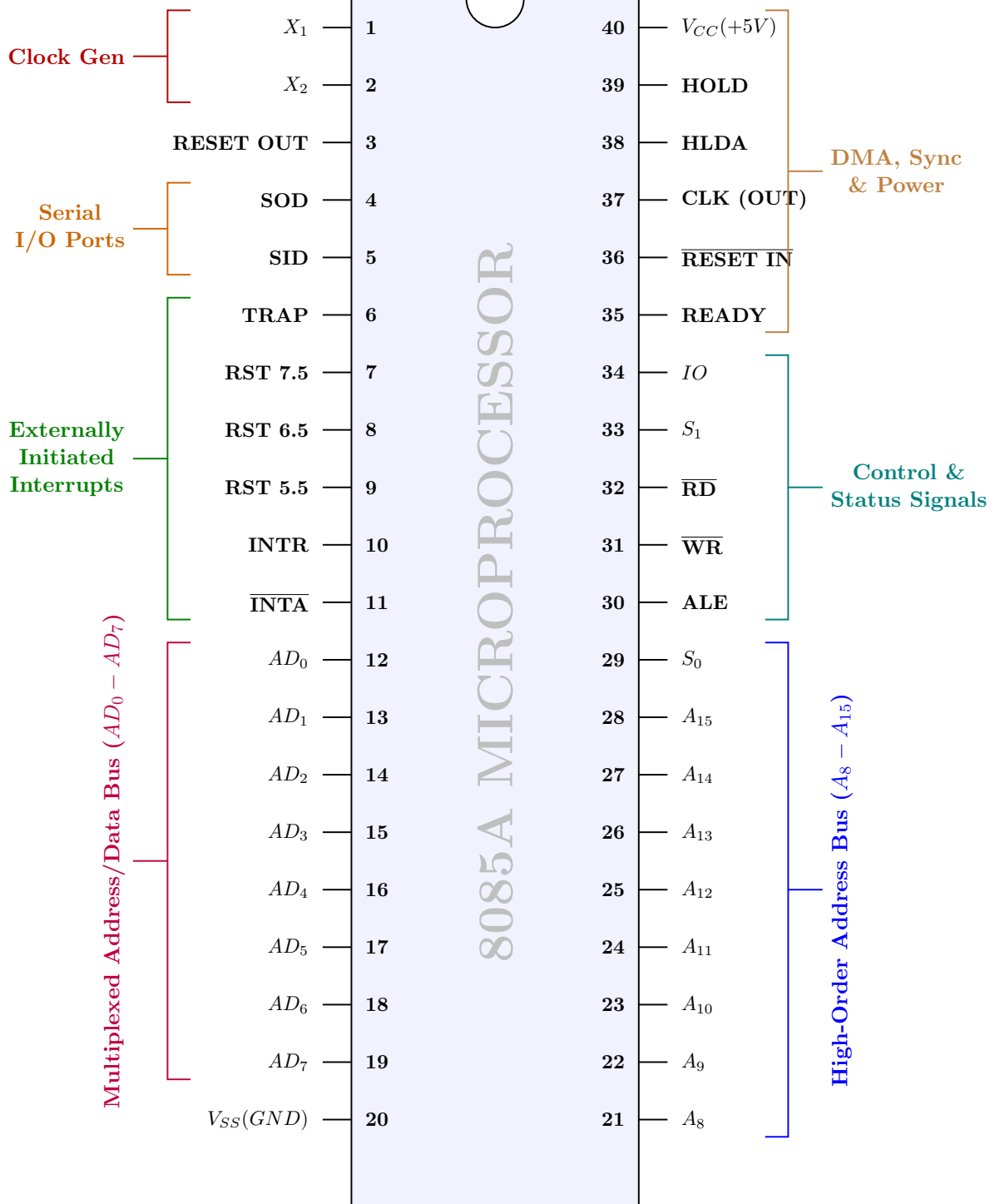
Sequential Systems & Microprocessors

B.E. Electrical Engineering - Part II Solutions

Microprocessor Architecture

Solution: Q1 (a) Functional Pin Diagram of 8085 Microprocessor

The Intel 8085 is an 8-bit microprocessor housed in a 40-pin Dual In-line Package (DIP). The pins are functionally grouped into Power/Clock, Interrupts, Serial I/O, Address Bus, Data/Address Multiplexed Bus, and Control/Status signals.



Solution: Q1 (b) Opcode Fetch Cycle for instruction MOV B,A

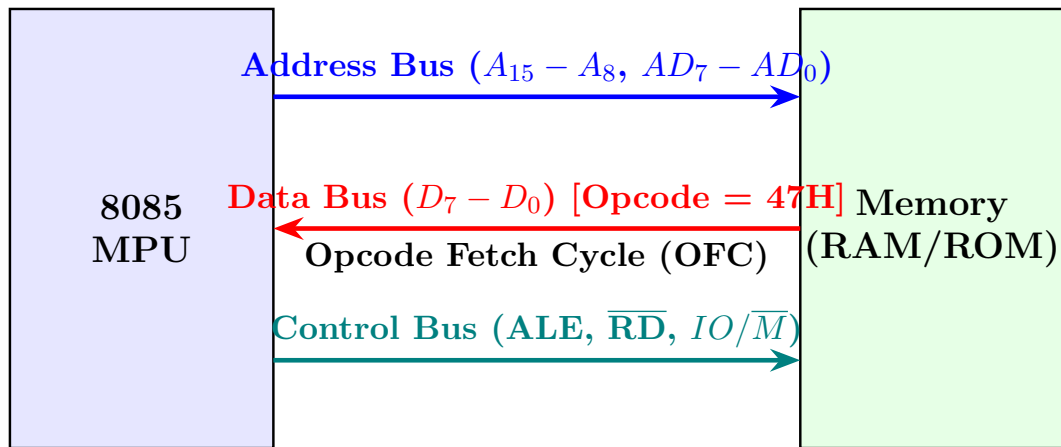
1. Instruction Overview

The instruction MOV B,A copies the contents of the Accumulator (A) into register B.

- **Opcode:** 47H (Hexadecimal) or 0100 0111 (Binary).
- **Size:** 1 Byte.
- **Machine Cycles:** 1 Cycle (Opcode Fetch Cycle).
- **T-States Required:** 4 T-States.

2. Functional Block Diagram

During the Opcode Fetch cycle, the microprocessor places the Program Counter (PC) address on the address bus and issues a Memory Read ($\overline{RD}$) control signal. The memory then places the opcode (47H) onto the data bus, which the microprocessor reads and decodes.

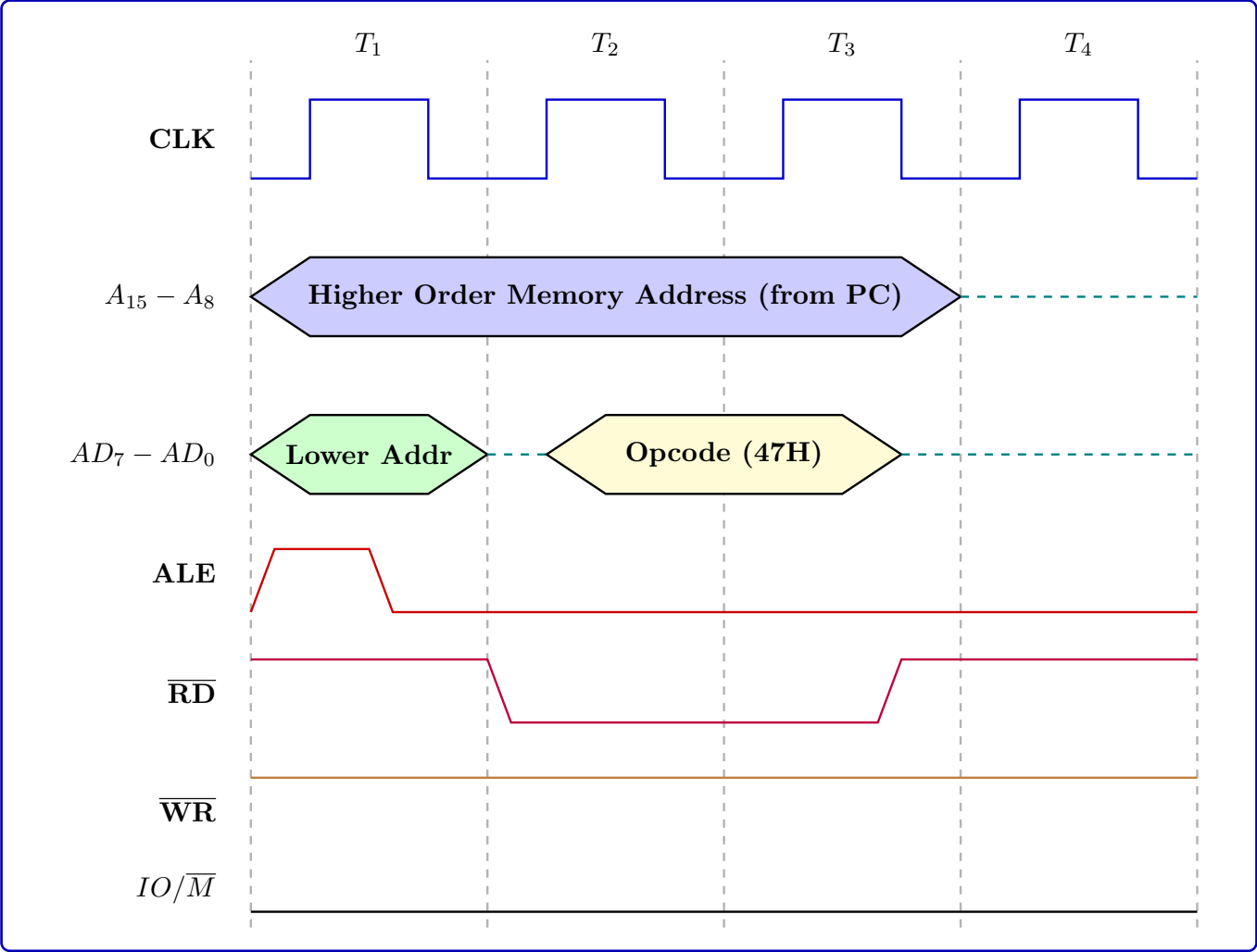


3. Timing Diagram Analysis

The Opcode Fetch Cycle consists of 4 T-states (T_1, T_2, T_3, T_4):

- **T₁ State:** The MPU places the higher byte of the PC on $A_{15} - A_8$ and the lower byte on $AD_7 - AD_0$. ALE goes HIGH to latch the lower address externally. $IO/\overline{M}$ goes LOW indicating a memory operation. Status signals $S_1 = 1, S_0 = 1$ identify it as an Opcode Fetch.
- **T₂ State:** ALE goes LOW. The $AD_7 - AD_0$ bus goes into a high-impedance state briefly. The $\overline{RD}$ signal goes LOW, enabling the memory chip to place the opcode on the data bus.
- **T₃ State:** The opcode (47H) is available on $AD_7 - AD_0$. The MPU reads this byte and $\overline{RD}$ goes HIGH.
- **T₄ State:** The MPU decodes and executes the instruction internally. The buses may hold unspecified data.

4. Timing Waveforms



Solution: Q2(a) Addressing Modes of 8085

Question: Define addressing modes and explain the four types used in the 8085: Implied, Immediate, Direct, and Indirect. Provide one assembly language example for each.

Definition of Addressing Mode:

An **addressing mode** specifies the method by which the operand of an instruction is identified. In simple words, it tells the microprocessor where the data is located or how the data is to be obtained.

In the 8085 microprocessor, the commonly used addressing modes are:

Implied, Immediate, Direct, and Indirect

1. Implied Addressing Mode

In implied addressing mode, the operand is not written explicitly in the instruction. The operand is already implied by the instruction itself.

Operand is understood automatically.

Example:

CMA

Explanation: The instruction CMA means **Complement Accumulator**. Here, the accumulator is not written as an operand, but it is understood that the operation is performed on accumulator A.

$$A \leftarrow \overline{A}$$

2. Immediate Addressing Mode

In immediate addressing mode, the data is given directly in the instruction itself.

Instruction contains the actual data.

Example:

MVI B, 45H

Explanation: This instruction loads the immediate 8-bit data 45H into register B.

$$B \leftarrow 45H$$

Here, 45H is the data itself, not an address.

3. Direct Addressing Mode

In direct addressing mode, the address of the operand is given directly in the instruction.

Instruction contains the memory address of the data.

Example:

LDA 4000H

Explanation: This instruction loads the accumulator with the data stored at memory location 4000H.

$$A \leftarrow [4000H]$$

Here, 4000H is the memory address.

4. Indirect Addressing Mode

In indirect addressing mode, the memory address of the operand is stored in a register pair. The instruction refers to the register pair, and the register pair points to the actual memory location.

Register pair contains the address of the operand.

Example:

LDAX B

Explanation: This instruction loads the accumulator with the data from the memory location whose address is stored in the *BC* register pair.

$$A \leftarrow [(BC)]$$

If,

$$BC = 2050H$$

then,

$$A \leftarrow [2050H]$$

Summary Table

max width=			
gray!25 Addressing Mode	Meaning	Example	Operation
Implied	Operand is implied by the instruction.	CMA	$A \leftarrow \overline{A}$
Immediate	Data is directly given in the instruction.	MVI B, 45H	$B \leftarrow 45H$
Direct	Memory address is directly given.	LDA 4000H	$A \leftarrow [4000H]$
Indirect	Address is stored in a register pair.	LDAX B	$A \leftarrow [(BC)]$

Addressing mode tells how the operand is specified in an instruction. The four addressing modes of 8085 are:

Implied, Immediate, Direct, Indirect

with examples:

CMA, MVI B, 45H, LDA 4000H, LDAX B

Solution: Q2(b) Time Delay Formula and Nested Loop Delay

Question: Derive the formula for calculating time delays in the 8085. Explain how a nested loop structure can be used to generate a delay of approximately 28.7 ms if the processor frequency is 2 MHz .

Basic Concept:

Each instruction in 8085 requires a certain number of **T-states**. One T-state is one clock period. If the clock frequency is f , then time for one T-state is:

$$T = \frac{1}{f}$$

Therefore, if an instruction or program takes N T-states, then delay is:

$$T_{\text{delay}} = \frac{N}{f}$$

where,

N = total number of T-states

and,

f = clock frequency

For 2 MHz clock frequency:

$$f = 2\text{ MHz} = 2 \times 10^6\text{ Hz}$$

$$T = \frac{1}{2 \times 10^6}$$

$$T = 0.5\text{ }\mu\text{s}$$

So,

$$1\text{ T-state} = 0.5\text{ }\mu\text{s}$$

Nested Loop Program

The required delay can be generated by the following nested loop:

MVI B,10H	7 T-states
LOOP2: MVI C,FFH	7 T-states
LOOP1: DCR C	4 T-states
JNZ LOOP1	10 T-states if jump is taken
DCR B	4 T-states
JNZ LOOP2	10 T-states if jump is taken

Here,

$$FFH = 255_{10}$$

and,

$$10H = 16_{10}$$

Important Point about JNZ:

For JNZ,

if jump is taken = 10 T-states

if jump is not taken = 7 T-states

So in the last iteration, we subtract:

$$10 - 7 = 3 \text{ T-states}$$

Step 1: Inner Loop Delay

The inner loop is:

LOOP1: DCR C

JNZ LOOP1

For one normal iteration:

$$DCRC + JNZ = 4 + 10 = 14 \text{ T-states}$$

The loop repeats 255 times. Therefore,

$$T_{L1} = (255 \times 14) - 3$$

$$T_{L1} = 3570 - 3$$

$$T_{L1} = 3567 \text{ T-states}$$

The instruction MVI C, FFH is executed once for each outer loop cycle:

$$T_{O1} = 7 \text{ T-states}$$

Therefore, total inner loop block delay is:

$$T_{\text{inner}} = T_{O1} + T_{L1}$$

$$T_{\text{inner}} = 7 + 3567$$

$$T_{\text{inner}} = 3574 \text{ T-states}$$

Step 2: Outer Loop Delay

The outer loop consists of:

Inner loop block + DCR B + JNZ LOOP2

For each normal outer iteration:

$$T_{\text{inner}} + DCRB + JNZLOOP2$$

$$= 3574 + 4 + 10$$

$$= 3588 \text{ T-states}$$

Since $B = 10H = 16$, the outer loop repeats 16 times.

So,

$$T_{L2} = (16 \times 3588) - 3$$

$$T_{L2} = 57408 - 3$$

$$T_{L2} = 57405 \text{ T-states}$$

The instruction `MVI B,10H` is outside the loop:

$$T_{O2} = 7 \text{ T-states}$$

Therefore,

$$T_{\text{total}} = T_{O2} + T_{L2}$$

$$T_{\text{total}} = 7 + 57405$$

$$T_{\text{total}} = 57412 \text{ T-states}$$

Step 3: Convert T-states into Time

At 2 MHz ,

$$1 \text{ T-state} = 0.5 \mu s$$

Therefore,

$$T_{\text{delay}} = 57412 \times 0.5 \mu s$$

$$T_{\text{delay}} = 28706 \mu s$$

$$T_{\text{delay}} = 28.706 \text{ ms}$$

$$T_{\text{delay}} \approx 28.7 \text{ ms}$$

Calculation Summary Table

max width=		
Step	Calculation	Result
Inner loop body	$DCRC + JNZ = 4 + 10$	14 T-states
Inner loop total	$(255 \times 14) - 3$	3567 T-states
Including <code>MVI C,FFH</code>	$3567 + 7$	3574 T-states
Outer loop one cycle	$3574 + 4 + 10$	3588 T-states
Outer loop total	$(16 \times 3588) - 3$	57405 T-states
Including <code>MVI B,10H</code>	$57405 + 7$	57412 T-states
Final delay at 2 MHz	$57412 \times 0.5 \mu s$	28.706 ms

The general delay formula is:

$$T_{\text{delay}} = \frac{\text{Total T-states}}{\text{Clock frequency}}$$

For the given nested loop:

$$\text{Total T-states} = 57412$$

At 2 MHz :

$$T_{\text{delay}} = 57412 \times 0.5\ \mu s = 28.706\ ms \approx 28.7\ ms$$

Thus, a nested loop using registers B and C can generate a delay of approximately $28.7\ ms$.

Microprocessor Interrupt Mechanism

Solution: Q3 (a) Internal Logic of 8085 Interrupts & SIM Instruction

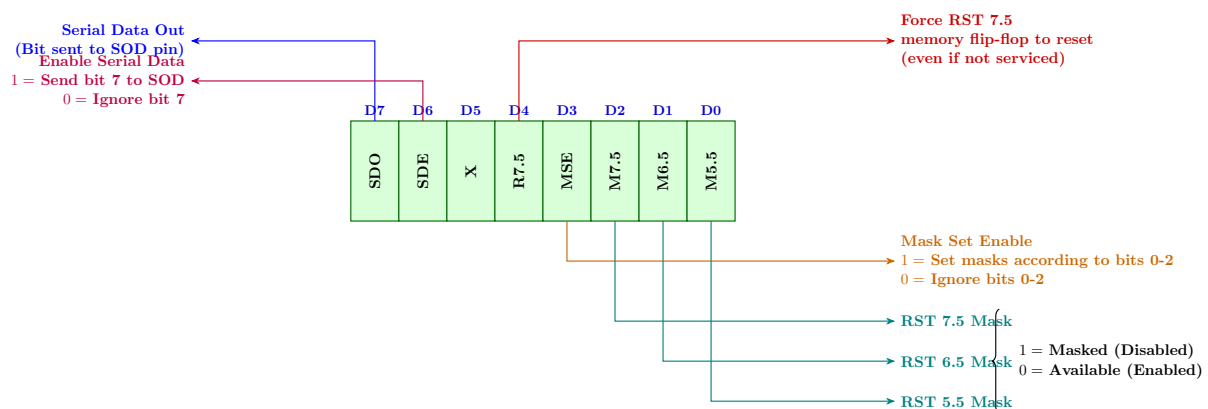
1. Internal Logic of 8085 Interrupts

The 8085 microprocessor supports 5 hardware interrupts: **TRAP**, **RST 7.5**, **RST 6.5**, **RST 5.5**, and **INTR**. The internal logic governing these interrupts operates on a strict priority and masking hierarchy:

- **Interrupt Priority:** The internal circuitry enforces the priority sequence: TRAP (Highest) → RST 7.5 → RST 6.5 → RST 5.5 → INTR (Lowest).
- **Interrupt Enable (IE) Flip-Flop:** This is the master switch for all maskable interrupts (RST 7.5, 6.5, 5.5, and INTR). It is enabled by the EI (Enable Interrupts) instruction and automatically disabled (reset) when an interrupt is acknowledged or by executing the DI instruction.
- **Individual Mask Flip-Flops:** RST 7.5, 6.5, and 5.5 each have individual mask flip-flops that provide a secondary level of control. These are manipulated exclusively by the SIM instruction.
- **Edge vs. Level Triggering:** TRAP is both edge and level-sensitive. RST 7.5 is strictly positive-edge triggered and features an internal memory flip-flop that remembers a pending pulse even if it is currently masked. RST 6.5, RST 5.5, and INTR are level-sensitive.

2. Interpretation of the SIM Instruction

The SIM (Set Interrupt Mask) instruction reads the 8-bit pattern currently residing in the Accumulator to perform two primary tasks: setting individual masks for the RST interrupts and outputting serial data.



How it enables/disables individual interrupts: To modify an interrupt mask, **Bit 3 (MSE)** MUST be set to 1. If MSE is 0, bits 0, 1, and 2 are ignored. When MSE is 1, the microprocessor copies the pattern of Bits 0, 1, and 2 into the mask flip-flops of RST 5.5, 6.5, and 7.5 respectively. A **logic 1** masks (disables) the corresponding interrupt, while a **logic 0** unmask (enables) it.

Subroutines and Stack Operations

Solution: Q3 (b) Mechanism of CALL

1. The Stack Memory and Stack Pointer (SP)

The Stack is a reserved area of standard RAM utilized for the temporary storage of parameters and return addresses. It operates on a **Last-In-First-Out (LIFO)** principle.

- The **Stack Pointer (SP)** is a dedicated 16-bit register inside the 8085 that always holds the memory address of the "top" of the stack.
- The stack typically **grows backwards** (downwards) in memory. As data is added, the SP decrements; as data is removed, the SP increments.

2. Mechanism of CALL and RET Instructions

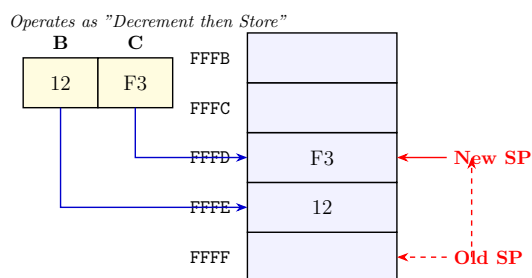
When a subroutine is executed, the main program must pause, redirect to the subroutine, and eventually return exactly where it left off.

- **CALL (16-bit Address):** When executed, the microprocessor must save its current place. It takes the 16-bit address of the *next* instruction (currently in the Program Counter, PC) and pushes it onto the stack (higher byte first, then lower byte). The SP is decremented by 2. Finally, the PC is loaded with the 16-bit address provided in the CALL instruction, causing execution to jump to the subroutine.
- **RET (Return):** Placed at the end of a subroutine. It pops the top two bytes from the stack (lower byte first, then higher byte) and loads them back into the PC. The SP is incremented by 2. Execution inherently resumes at the exact instruction following the original CALL.

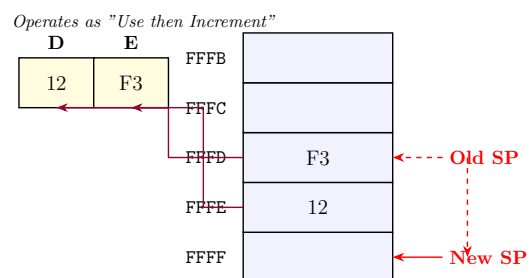
3. Preserving Registers using PUSH and POP

Subroutines frequently manipulate registers. To prevent overwriting data needed by the main program, original register contents are preserved on the stack.

PUSH Operation (Store)



POP Operation (Retrieve)



- **PUSH R_p (e.g., PUSH B):** The SP is decremented by 1, and the higher-order register (B) is copied into that memory location. The SP is decremented again by 1, and the lower-order register (C) is copied.
- **POP R_p (e.g., POP D):** The contents of the memory location pointed to by SP are copied into the lower-order register (E), and SP is incremented by 1. The contents of the new SP location are copied into the higher-order register (D), and SP is incremented again.

Crucial Rule: Because of the LIFO structure, registers pushed onto the stack must be popped off in the exact **reverse order** (e.g., PUSH B, PUSH D → POP D, POP B) before executing RET to prevent crashing the program.

Microprocessor Instructions & Debugging

Solution: Q4 (a) Execution of 8085 Instructions

- (i) **MVI M, data:** (Move Immediate to Memory) The 8-bit data specified in the instruction is stored directly into the memory location pointed to by the 16-bit address held in the H-L register pair.
- (ii) **LDA address:** (Load Accumulator Direct) The 8-bit contents of the memory location specified by the 16-bit address provided in the instruction are copied into the Accumulator (Register A).
- (iii) **XTHL:** (Exchange Stack Top with H-L) The contents of the L register are exchanged with the contents of the memory location pointed to by the Stack Pointer (SP). Simultaneously, the contents of the H register are exchanged with the contents of the memory location at (SP + 1).
- (iv) **DAA:** (Decimal Adjust Accumulator) Executed after an arithmetic addition of BCD numbers, it converts the binary result in the Accumulator back into a valid 2-digit Binary Coded Decimal (BCD) format. It adds 06H if the lower nibble > 9 or AC flag is set, and adds 60H if the upper nibble > 9 or CY flag is set.
- (v) **CMP M:** (Compare Memory with Accumulator) The contents of the memory location pointed to by the H-L register pair are subtracted from the Accumulator. The Accumulator's data remains unchanged, but the status flags (Z, CY, S, P, AC) are modified to reflect the result (e.g., if $A < M$, CY=1; if $A = M$, Z=1).
- (vi) **SPHL:** (Copy H-L pair to Stack Pointer) The 16-bit contents of the H-L register pair are copied into the Stack Pointer (SP) register ($SP \leftarrow HL$). This is typically used to initialize the stack pointer dynamically.

Solution: Q4 (b) Debugging 8085 Programs

(i) Delay Loop Program

- **Original Code:** MVI C, FFH → LOOP: INR C → JNZ LOOP → HLT
- **Bug & Justification:** The intention is to create a delay loop executing 255 (FFH) times. However, the INR C instruction increments FFH (255) to 00H (0) on the very first pass. This immediately sets the Zero Flag ($Z = 1$). The JNZ (Jump if Not Zero, i.e., $Z = 0$) condition fails immediately, and the loop terminates after just one iteration.
- **Corrected Code:**

```
MVI C, FFH
LOOP: DCR C      ; Decrement C instead of incrementing
      JNZ LOOP   ; Will now loop 255 times until C = 00H
      HLT
```

(ii) Stack Operations Program

- **Original Code:** LXI SP, FFFFH → PUSH B → PUSH D → POP B → POP D → HLT
- **Bug & Justification:** The stack operates on a **Last-In-First-Out (LIFO)** principle. The code pushes the B-C pair, then the D-E pair. Therefore, the data belonging to D-E is at the top of the stack. Executing POP B first retrieves the D-E data into the B-C registers, effectively swapping their contents instead of restoring them.
- **Corrected Code:**

```
LXI SP, FFFFH
PUSH B
PUSH D
POP D      ; Pop in the exact reverse order of Push
POP B
HLT
```

Memory and I/O Interfacing

Solution: Q4 (c) Interfacing 2K Memory at 2000H and I/O at 07H

1. Address Decoding Logic Formulation:

- **Memory IC (2K size):** 2K bytes require 11 address lines ($A_{10} - A_0$). The base address is 2000H.

$$2000H = \underbrace{0 \ 0 \ 1 \ 0 \ 0}_{A_{15}-A_{11}} \ \underbrace{0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0}_{A_{10}-A_0}$$

To select the chip ($\overline{CS_{MEM}}$ active low), the decoder must activate when $A_{15} = 0, A_{14} = 0, A_{13} = 1, A_{12} = 0, A_{11} = 0$, and $IO/\overline{M} = 0$.

$$\overline{CS_{MEM}} = \overline{A_{15}} \cdot \overline{A_{14}} \cdot A_{13} \cdot \overline{A_{12}} \cdot \overline{A_{11}} \cdot \overline{IO/\overline{M}}$$

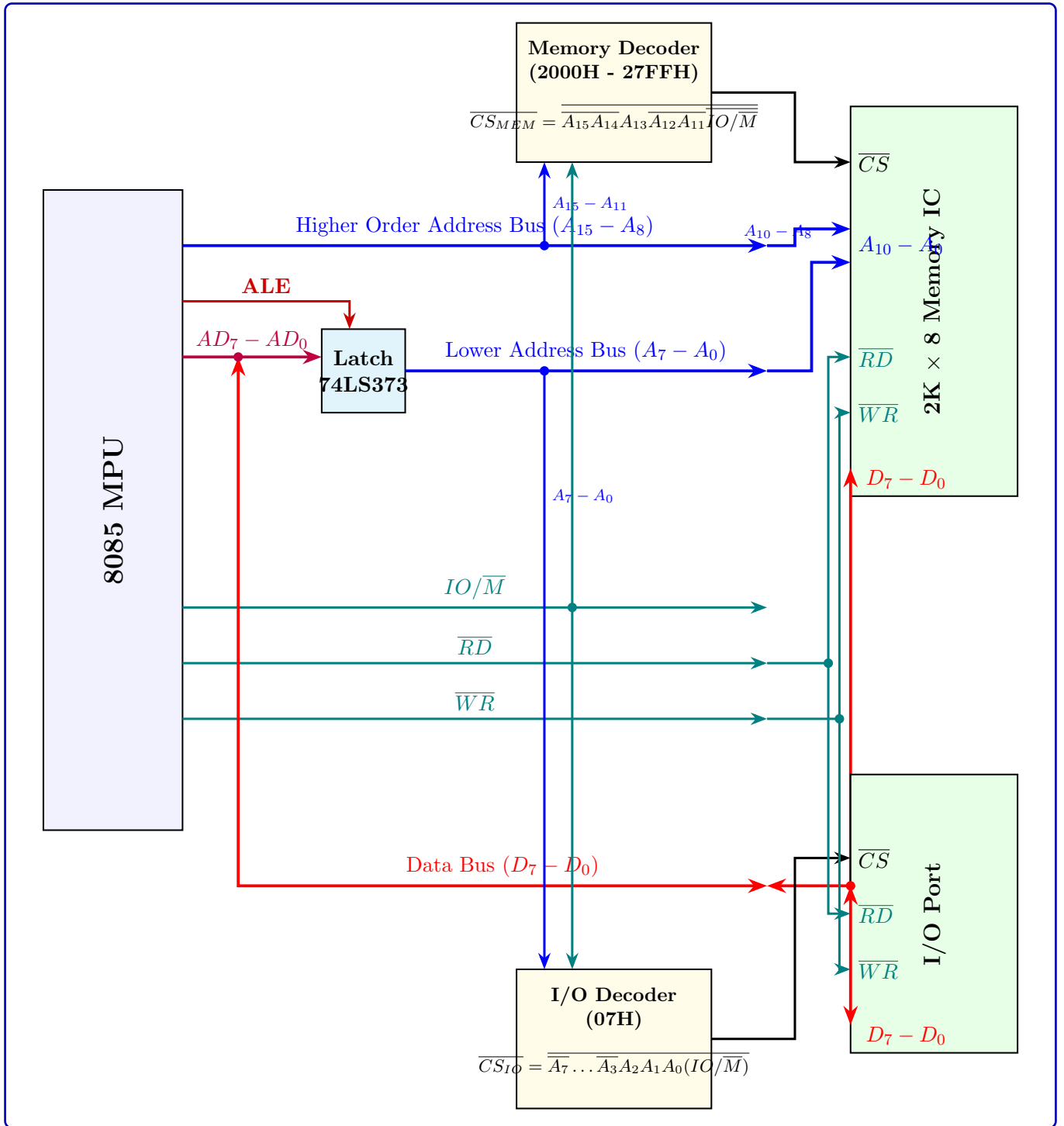
- **I/O Port (Address 07H):** The 8-bit port address is decoded from $A_7 - A_0$.

$$07H = \underbrace{0 \ 0 \ 0 \ 0 \ 0}_{A_7-A_3} \ \underbrace{1 \ 1 \ 1}_{A_2-A_0}$$

To select the chip ($\overline{CS_{IO}}$ active low), it activates when $A_{7-3} = 0, A_{2-0} = 1$, and $IO/\overline{M} = 1$.

$$\overline{CS_{IO}} = \overline{A_7} \cdot \overline{A_6} \cdot \overline{A_5} \cdot \overline{A_4} \cdot \overline{A_3} \cdot A_2 \cdot A_1 \cdot A_0 \cdot (IO/\overline{M})$$

2. Schematic Interfacing Diagram (Using Strict Perpendicular Routing):



Solution: Q5(a) 8085 Program to Add Two 8-bit Data

Question: Write an assembly language program in 8085 to add two 8-bit data, the result being 16-bit.

Assumption:

Let the two 8-bit numbers be stored at memory locations:

2500H and 2501H

The 16-bit result will be stored as:

2502H = Lower byte of result

2503H = Higher byte of result

Since the addition of two 8-bit numbers may generate a carry, the result may become 9-bit. Therefore, the carry is stored as the higher byte of the 16-bit result.

Memory Arrangement

max width=	Memory Location	Content
	2500H	First 8-bit number
	2501H	Second 8-bit number
	2502H	Lower byte of sum
	2503H	Higher byte of sum, that is carry byte

Logic Used

The first number is loaded into the accumulator. Then the second number is added to it. The accumulator stores the lower 8 bits of the result.

If carry is generated, the carry flag becomes 1. This carry is converted into either 00H or 01H and stored as the higher byte.

16-bit result = Carry byte : Sum byte

Assembly Language Program

max width=		
Step	Instruction	Comment
1	LXI H, 2500H	Load HL pair with address of first data.
2	MOV A, M	Copy first number from memory into accumulator.
3	INX H	HL points to second data at 2501H.
4	ADD M	Add second number with accumulator.
5	STA 2502H	Store lower byte of result at 2502H.
6	MVI A, 00H	Clear accumulator to prepare carry byte.
7	ADC A	Add accumulator with itself and carry. So A becomes 00H or 01H.
8	STA 2503H	Store higher byte of result at 2503H.
9	HLT	Stop the program.

Program in Compact Form

8085 Program
LXI H,2500H
MOV A,M
INX H
ADD M
STA 2502H
MVI A,00H
ADC A
STA 2503H
HLT

Why ADC A is used?

After ADD M, the carry flag may be:

$$CY = 0 \quad \text{or} \quad CY = 1$$

Then,

MVI A,00H

makes:

$$A = 00H$$

Now,

ADC A

means:

$$A \leftarrow A + A + CY$$

So,

$$A = 00H + 00H + CY$$

Therefore,

$$A = 00H \quad \text{if } CY = 0$$

$$A = 01H \quad \text{if } CY = 1$$

Hence, A becomes the higher byte of the 16-bit result.

Example Verification

Suppose:

$$[2500H] = F2H$$

$$[2501H] = 34H$$

Then,

$$F2H + 34H = 126H$$

So,

$$2502H = 26H$$

$$2503H = 01H$$

The lower byte of the 16-bit sum is stored at:

$$2502H$$

The higher byte, that is the carry byte, is stored at:

$$2503H$$

Thus, the program adds two 8-bit data and stores the result as a 16-bit value.

Solution: Q5(b) 8085 Program to Convert 7FH to Binary

Question: Write an assembly language program in 8085 to convert 7FH to binary.

Required Conversion

The hexadecimal number is:

$$7FH$$

Now,

$$7H = 0111_2$$

and,

$$FH = 1111_2$$

Therefore,

$$7FH = 01111111_2$$

Aim of the Program

The program will convert 7FH into its 8-bit binary form and store each bit separately in memory. We store:

$$0 \text{ as } 00H$$

$$1 \text{ as } 01H$$

The output bits will be stored from memory location:

$$3000H \text{ to } 3007H$$

Memory Output Arrangement

max width=	Memory Location	Stored Value	Meaning	
	3000H	00H	First bit = 0	
	3001H	01H	Second bit = 1	
	3002H	01H	Third bit = 1	
	3003H	01H	Fourth bit = 1	
	3004H	01H	Fifth bit = 1	
	3005H	01H	Sixth bit = 1	
	3006H	01H	Seventh bit = 1	
	3007H	01H	Eighth bit = 1	

So the binary output is:

$$01111111$$

Logic Used

The instruction RLC rotates the accumulator left. The most significant bit D_7 goes into the carry flag and also comes to D_0 . Therefore, by repeatedly using RLC, we can test each bit of the accumulator one

by one. The PDF explains RLC as rotate accumulator left, where bit 7 goes to bit 0 and the carry flag.
 :contentReference[oaicite:2]index=2
 For each rotation:

$$CY = 0 \Rightarrow \text{store } 00H$$

$$CY = 1 \Rightarrow \text{store } 01H$$

Since the number has 8 bits, the loop is repeated 8 times.

Assembly Language Program

max width=		
Step	Instruction	Comment
1	MVI A,7FH	Load hexadecimal number 7FH into accumulator.
2	LXI H,3000H	HL points to first output memory location.
3	MVI B,08H	Counter for 8 bits.
4	LOOP: RLC	Rotate accumulator left. MSB goes to carry flag.
5	MVI M,00H	Assume current bit is 0. Store 00H.
6	JNC NEXT	If carry is 0, go to NEXT.
7	MVI M,01H	If carry is 1, store 01H.
8	NEXT: INX H	Move to next memory location.
9	DCR B	Decrease bit counter.
10	JNZ LOOP	Repeat until all 8 bits are stored.
11	HLT	Stop the program.

Program in Compact Form

8085 Program
MVI A,7FH
LXI H,3000H
MVI B,08H
LOOP: RLC
MVI M,00H
JNC NEXT
MVI M,01H
NEXT: INX H
DCR B
JNZ LOOP
HLT

Stepwise Trace

Initial value:

$$A = 7FH = 01111111_2$$

max width=				
gray!25 Rotation	Bit Sent to Carry	Stored Value	Memory Location	
1	0	00H	3000H	
2	1	01H	3001H	
3	1	01H	3002H	
4	1	01H	3003H	
5	1	01H	3004H	
6	1	01H	3005H	
7	1	01H	3006H	
8	1	01H	3007H	

The hexadecimal number:

$$7FH$$

is converted to binary as:

$$7FH = 01111111_2$$

The program stores this binary result bit by bit from memory location:

$$3000H \text{ to } 3007H$$

$$3000H = 00H, \quad 3001H = 01H, \quad 3002H = 01H, \quad \dots, \quad 3007H = 01H$$